Object Oriented Programming Exam

The exam has five exercises:

- You have to create the code to solve these exercises.
- The exercises are chained. Each exercise (partially) builds on the solution of the previous exercises.
- Do not create duplicated classes. Modify the same class structure as needed.
- Submit the global solution for all the exercises.

Instructions:

- Students are allowed to use notes and material they want (physical or electronic).
- Students are not allowed to query the web, use generative AI tools, or automatic code generation.
- Students can use any IDE they want (e.g., Visual Code, IntelliJ) as long as no generative AI tools are activated.

Submission:

- Place all Java classes in a folder called *src* and compress it into **a zip file**. Submit this zip file via Digital Exam.

Exercise 1

A library system needs to manage books and their locations in the library. Your task is to create the foundational classes for this system.

Part (a)

Create a Java class `BookLocation` to store the `section` (String) and `shelf` (int) within the section of a book. Once initialized, these attributes cannot be modified. Write code for this class, including its fields, constructor, and the required methods.

Part (b)

Create a Java class `Book` to represent a book in the library. Objects of this class should contain the following attributes:

- A `String ISBN` (unique identifier for the book). The ISBN must be initialized when the object is created and **cannot** be modified later.
- A `String title` to represent the book's title. This value must be initialized and **cannot** be modified later.
- A `String author` to represent the book's author. This value must be initialized and **cannot** be modified later.
- A `BookLocation` to represent the location of the book in the library. The location is initialized when the object is created.

Write the code for this class, including its fields and constructor. Provide the corresponding getters and setters. Override the `toString` method to provide a meaningful string representation that includes all attributes.

Exercise 2

Extend the library system to handle specific types of books: textbooks and novels. Create two classes that inherit from the `Book` class.

Requirements

1. **Class** `TextBook`:
    - Inherits from the `Book` class.
    - Contains an additional attribute:
        - `String subject`: Represents the subject of the textbook (e.g., Mathematics, Physics).
    - The `subject` value must be initialized when the `TextBook` object is created and cannot be modified later.
    - The location of a `TextBook` object **can not be changed** once the object is created.
2. **Class** `Novel`:
    - Inherits from the `Book` class.
    - Contains an additional attribute:
        - `String genre`: Represents the genre of the novel (e.g., Mystery, Fantasy).
    - The `genre` value must be initialized when the `Novel` object is created and cannot be modified later.
    - The location of a `Novel` object **can be changed** after the object is created.

Both `TextBook` and `Novel` classes must override the `toString` method to include all attributes (inherited and specific).

Exercise 3

The library needs a centralized system to manage all its books. Your task is to create a new class, `Library`, to manage a collection of `Book` objects.

Requirements

1. **Class** `Library`:
    - The class should include an **empty constructor** that initializes an empty collection to hold `Book` objects.
    - Internally, use a suitable collection type (e.g., `List`, `Set`, `Map`).
    - Provide the following methods:
        - `void addBook(Book book)`:
            - Adds a `Book` object to the collection.
            - If another book with the same `ISBN` already exists, throw a `DuplicateBookException` (custom exception).
        - `void removeBook(Book book)`:
            - Removes a `Book` object.
            - If the book is not found, do nothing.
        - `Collection<Book> findBooksAtLocation(BookLocation location)`:
            - Searches for `Book` objects placed in a given book location.
            - Returns an empty collection if no match is found.

Exercise 4

The library allows users to search for books in different ways. Implement the **Strategy Design Pattern** to enable different search strategies.

Requirements

1. Create an interface `SearchStrategy` with a method :

   ```
   Collection<Book> search(String query, Collection<Book> books)
   ```

2. Implement at least two concrete search strategies:

   - `SearchByAuthor` – Searches for books by author name.
   - `SearchByTitle` – Searches for books by title.

3. Modify the `Library` class to accept a `SearchStrategy` and use it to search for books.

Exercise 5

The library wants to track books placed in a specific location. Implement the **Observer Design Pattern** to keep track of books added or removed from the specific `BookLocation`.

Requirements

1. **Interface** `LocationTracker`:
   - Represents any tracker that monitors books at a specific location.
   - Contains the following methods:
     - `void update(Book book, boolean isAdded)`:
     - `Collection<Book> getBooks()`:
2. `BookLocation` is an observer/listener and, then, implements `LocationTracker`. The `Library` is the subject.
3. Write UnitTests to test the implementation of the Observer Design Pattern.